

Assignment 2: Dynamic Memory Allocator

C S 429, Spring 2025

Unique ID: 50815, 50820, 50825, 50830, 50835, 50840, 50845, 50850, 50855, 50860, 50865, 50870

Lead TAs: Jay Upadhayaya, Andy Lieou, Hanyu Wei

Assigned: Friday, 7 February 2025 00:00 CT

Week 1 Due: Thursday, 13 February 2025 23:59 CT

Week 1 Last Possible Hand In: Saturday, 15 February 2025 23:59 CT

Week 2 Due: Thursday, 20 February 2025 23:59 CT

Week 2 Last Possible Hand In: Saturday, 22 February 2025 23:59 CT

1 Introduction

In this lab you will be writing a dynamic storage allocator for C programs, i.e., your own version of the `malloc` and `free` routines. Explore the design space creatively. You must implement an allocator that is correct, space-efficient, and high-throughput.

2 Logistics

You are required to complete two submissions: week one due Thursday, 13 February 2025 and week two due Thursday, 20 February 2025. The grading details for each submission are given in Section 6.

From this lab onwards, **the TAs are not allowed to read your code or do any direct debugging during office hours**. Come to office hours prepared to explain your implementation strategies. You are welcome to submit a code review should you have trouble with **specific** bugs. We will reject code reviews that do not demonstrate enough debugging effort from the student-side.

Start early enough to get things done before the due dates. Assume things will not go according to plan, and so you must allow extra time for heavily loaded systems, dropped internet connections, corrupted files, traffic delays, minor health problems, *force majeure*, etc.

Unless otherwise stated, every project in C S 429 is an individual project. Before you begin, please take the time to review and abide by the course policy on academic integrity at: <https://www.cs.utexas.edu/academics/conduct>.

All hand-ins are electronic. The testing for your code is automated, using OS and compiler configurations similar to those on the UTCS Linux machines. *Test your code thoroughly on a UTCS Linux Lab machine before submitting it.*

Any clarifications or corrections for this lab will be posted on Ed. You should also ask your questions on that forum. Unless it's an implementation-specific question (i.e., private to instructors), please post it on Ed publicly so that students with similar questions can benefit as well.

3 Download and setup

First, go to our GitHub Classroom page and create a copy of the assignment: <https://classroom.github.com/a/VKg19tbj>. This will automatically create an assignment repository specific to you on GitHub. Now clone this created repository on a UTCS lab machine using the command `git clone your-repo-url`, remembering to substitute the actual URL for the placeholder `your-repo-url`. Ask a TA for help if you are having trouble with these steps.

Run `./install.sh` in this directory. If you get a “permission denied” error message, run the command `chmod +x install.sh` in the working directory to give execution permissions. Running `install.sh` will install some Python libraries required to successfully run the test script. Additionally, update the file `umalloc.c` with your name and UT EID as specified, and save your changes.

The only files you will modify from here on are `umalloc.h`, `umalloc.c`, and `check_heap.c`. You will turn in your assignment as you did in Assignment 1, by linking your GitHub repository to Gradescope.

The file `runner.c` contains source for a driver program that allows you to evaluate the performance of your solution. Use the command `make` to generate the driver program, and run it with the command `./runner`. The `-h` flag displays the various command-line arguments that you can present to the driver.

Important: Do this assignment **only on the UTCS Linux machines**. Do not do any development on your local desktop or laptop. You may, of course, `ssh` into these machines from your local machine as you did for CI Lab.

4 Details of the assignment

Your dynamic storage allocator will consist of the following three functions declared in `umalloc.h` and implemented in `umalloc.c`.

```
int    uinit(void);
void*  umalloc(size_t size);
void   ufree(void* ptr);
```

The supplied file `umalloc.c` provides a handful of useful data structure declarations and utility methods, plus placeholders for these three functions. Using this file as a starting template, modify these functions (and optionally define other `static` functions), so that they obey the following semantics.

- **uinit:** Before calling `umalloc` or `ufree`, the application program (i.e., the trace-driven driver program that you will use to evaluate your implementation) calls `uinit` to perform any necessary initializations, such as allocating the initial heap area. The return value should be `-1` if there was a problem in performing the initialization, `0` otherwise.
- **umalloc:** The `umalloc` routine returns a pointer to an allocated block payload of at least `size` bytes. The entire allocated block should lie within the heap region and should not overlap with any other allocated chunk.

We will be comparing your implementation to the version of `malloc` supplied in the standard C library (`libc`). Since the standard `malloc` on 64-bit Linux systems returns payload pointers that are aligned to 16 bytes, your `umalloc` implementation must do likewise and always return 16-byte aligned payload pointers.

- `ufree`: The `ufree` routine frees the block pointed to by `ptr`. It returns nothing. This routine is guaranteed to work only when the passed pointer `ptr` was returned by an earlier call to `umalloc` and has not yet been freed.

These semantics match the the semantics of the corresponding standard `malloc` and `free` routines. Type `man malloc` in the Linux shell for complete documentation.

4.1 Heap consistency checker

Dynamic memory allocators are notoriously tricky beasts to program correctly and efficiently. They are difficult to program correctly because they can involve untyped pointer manipulation. We therefore require you to write a heap checker that scans a free list and checks for its consistency. The interface declared in the file `check_heap.h` is `check_bin(mem_block_header_t *)`. You will implement this routine in the file `check_heap.c`.

4.1.1 Required checks

You must implement the following four checks.

1. Check that the free block list conforms to the list ordering you have chosen. This check is not required if your list uses a random ordering.
2. Check whether any free memory blocks overlap with each other. They should not.
3. Check that each memory block is 16-byte aligned.
4. Check that every block on the free list is marked as free. Having allocated blocks on the free list indicates that you have an implicit free list.

Your implementation of `check_bin()` must check for the *four* invariants or consistency conditions listed above. It should return `HEAP_SUCCESS` if and only if your heap is consistent, and `HEAP_FAILURE` if not. **Do not use assertions.**

To begin, we recommend implementing a heap checker that prints the contents of your free list for you. This will be very helpful for the short traces. Then, as your implementation matures, begin to implement the heap checks shown above.

4.1.2 Testing heap checker

In `check_heap.c`, there are two global variables `order` and `circular`. Set these variables to the correct values depending on *your* heap design. These global variables affect the behavior of the autograder. **Do not forget to set them!**

The file `heap_runner.c` contains source code for a driver program that will run your `check_bin()` against a series of free lists to ensure you are correctly testing for the required checks. Use the command `make heap_runner` to generate a driver program and run it with the command `./heap_runner`. The `-h` flag displays the various command-line arguments that you can present to this driver.

There are some heap designs that these tests do not work for. Such implementations will be manually graded by the TAs *after* the deadline.

4.1.3 Optional checks

Here is a list of additional checks you could implement if you wanted to.

- Is every free block on the free list?
- Are all bytes obtained from the operating system through `sbrk()` calls accounted for in either an allocated block or a free block?
- For coalescing: are there contiguous free blocks that somehow escaped coalescing?

If you would like to implement more checks, you are not limited to the listed suggestions, nor are you required to check all (or any) of them beyond the required ones. Many of the suggestions, especially those that require checking allocated blocks, may require additional variables or helper functions to implement. *Note that you will not earn any extra credit for implementing additional checks.* They are purely intended to help you put your heap through more stringent tests for structural integrity.

4.2 Support routine

The supplied object `csbrk.o` file provides a thin wrapper (aka a “shim”) around the standard Linux system call `sbrk` that provides a chunk of memory to your allocator. You can do a `man sbrk` to learn more about this routine. However, you **must not** call `sbrk()` directly, but only through the provided wrapper.

The declared interface for `csbrk` is `void* csbrk(intptr_t increment);`. The argument `increment` is the amount of memory your allocator is requesting from the operating system. The returned value is a pointer to the starting address of a contiguous range of memory of at least this size. A return value of `NULL` indicates an error condition. Keep a few things in mind as you use this call.

- This is a wrapper around a *system call*. Therefore, do not call it frequently.
- Ask for a decent amount of space each time, e.g., a small multiple of `PAGESIZE` (a macro defined in `csbrk.h`).
- While each call to `csbrk` returns a contiguous chunk of memory, successive calls to `csbrk` are not guaranteed to return adjacent chunks of memory. *This is a significant change to the underlying assumptions of many memory allocators available through a web search; in particular, any allocator using an implicit free list will almost certainly fail.*

4.3 The trace-driven driver program

The driver program `runner` created from the source file `runner.c` tests your `umalloc.c` package for correctness, space utilization, and throughput. The driver program is controlled by a set of *trace files* that are included in the `traces` subdirectory. Each trace file contains a sequence of allocate and free operations of various sizes that instruct the driver to call your `umalloc` and `ufree` routines. The driver and the trace files are the same ones we will use when we grade your final submission.

The `runner -h` command will display the various command-line arguments that you can present to the driver. These include several useful facilities, such as the ability to single-step through the trace. The file `traces/README` contains a lot of useful information about the structure of the trace files.

4.4 Debugging with GDB

GDB is a great tool for debugging your code, finding segmentation faults, and stepping through your routines. The provided Makefile provides two targets to compile your code.

1. The `debug` target uses the `-O0` flag, which tells the compiler not to optimize your code. This makes it possible to debug using GDB. Use this version at the beginning to firm up your code.
2. The `deploy` target uses the `-O2` flag, which the compiler to optimize your code. This has the side effect of optimizing out variables, making it harder to see what's going on in GDB. This is the version that you will use for performance testing, and this is the target that the autograder will build and run.

Because the traces are so long, it can be useful to use **conditional breakpoints** to break at a specific line in the trace. For example, if your `umalloc` fails the correctness check after `umalloc(2048)`, you can use the command `(gdb) b umalloc if size == 2048` to set a conditional breakpoint for this case.

4.5 Programming rules

- You must update line 7 of `umalloc.c` as indicated.
- You must not change any of the interfaces in `umalloc.c`.
- You must not implement an implicit free list. Allocated blocks must be removed from the free list. Note that this means there must not be any allocated blocks in your free list. *Implementing an implicit free list will earn you an automatic zero on the assignment.*

If you have questions, please ask the TAs before the assignment deadline.

- You must not implement a bump allocator. You must not implement the allocator described in §8.7 of K&R2e (*The C Programming Language* textbook). *Implementing either one will earn you an automatic zero on the assignment.*

If you have questions, please ask the TAs before the assignment deadline.

- You must not invoke any memory-management related library calls or system calls. This forbids the use of the standard `malloc`, `calloc`, `free`, `realloc`, `sbrk`, `brk` or any variants of these calls in your code. Similarly, you may not copy any code from the memory-management related library calls or system calls.
- You must not implement a single free list approach. You must utilize binning, with multiple free lists (bin count ≥ 4), where each bin satisfies a different range of allocation request sizes.
- You must not define any global or `static` variables of derived data types such as arrays, structs, trees, or lists in your program. However, you *are* allowed to declare global or `static` variables of the basic data types and of pointers.
- For consistency with the system `malloc` package, which returns blocks aligned on 16-byte boundaries, your allocator must always return payload pointers that are aligned to 16-byte boundaries. The driver will enforce this requirement for you.

5 Handing in your assignment

Submit the files `umalloc.h`, `umalloc.c`, and `check_heap.c` to Gradescope. **These files must be at the top level of your git repository.** The autograder for this assignment will look for `./umalloc.h`, not for `./MM-lab/umalloc.h`. Prior to submitting, please format your code submission (the shortcut for this on VSCode is `Alt+Shift+f`).

6 Evaluation

This assignment is worth 8% of the total course grade.

- The week one submission will contribute 4%, and you will receive these points if all of the following conditions hold:
 - (a) Your uploaded `check_heap.c` file implements a working heap consistency checker following the instructions given in Section 4.1.
 - (b) Your uploaded `umalloc.c` file passes the tests in the four files `short1.rep`, `short2.rep`, `short1-bal.rep`, and `short2-bal.rep`. *Note that passing means correctness only.*
 - (c) Your solution **does not implement a bump allocator** and **implements an explicit free list**.
 - (d) Your solution implements a **binned free list** approach (see Section 4.5 for more details).
- The week two submission will contribute 4%. Your code must compile without error. The correctness of your solution will be determined by the supplied traces.

You will receive **zero points** if you break any rules or if your code is buggy and crashes the driver. Otherwise, your grade will be calculated as follows:

- (a) *Correctness (15%)*. You will receive full points for correctness only if your solution passes all of the traces performed by the driver program. You will receive partial credit for each correct trace. You **must** pass all traces to receive **any** utilization or throughput points.
- (b) *Space utilization (55%)*. The autograder will determine and report the average space utilization of your solution. An average utilization of 75% will earn full credit for space utilization. **An average utilization under 45% will earn zero credit for utilization. Coalescing, splitting, and free list binning is required to receive credit for space utilization.**
You can earn additional points for an average utilization over 75%. All submissions will be ranked on a public (but anonymous) leaderboard, and the top performers will receive rewards based on their placement.
- (c) *Throughput (15%)*. The autograder will determine and report the average throughput of your solution. An average throughput of 1400 **in the autograder** will earn full credit for throughput.¹ There will **not** be additional credit for higher throughput.
Note that some CS Unix machines (specifically, the cereal machines) have faster processors, and therefore will have throughput values that differ from the values returned by Gradescope. The throughput score on Gradescope is final.
- (d) *Style (5%)*. Style points will be graded manually.

¹The unit for throughput are unspecified. All that matters is that higher values are better.

- (i) Your code must be modularized into functions and use as few global variables as possible. You will need to use some global variables (such as a pointer to the head of your free list), but only add more if they are absolutely necessary to implement your solution.
 - (ii) Any changes to the provided data structures or helper functions must be documented.
 - (iii) Any new data structures or helper functions you introduce must be documented.
 - (iv) Outside of the above guidelines, you just need to use a consistent style throughout your code. For example, we don't enforce any specific variable name convention, but you should be consistent with whatever convention you choose.)
 - (v) Our style grading is not as strict as CS 314. In general, we want your code to be easy to follow and understand.
If you have any questions, please ask the TAs before the assignment deadline.
- (e) *Writeup (10%).* You must answer the following questions in an assignment writeup. This will be due at the same time as the final submission, and will be submitted through a separate Gradescope assignment. Responses should be succinct yet descriptive. This component will be graded manually. There is a word limit enforced for each question to ensure conciseness. **You will be penalized for exceeding the word limit.**
- (i) Describe your free block insertion policy. (*50 words max.*)
 - (ii) Describe how you chose to split allocated blocks. Always? Sometimes? Never? Which end? (*50 words max.*)
 - (iii) Describe how you select which free block to allocate. What placement strategy are you using? (*50 words max.*)
 - (iv) Describe your coalescing strategy. (*50 words max.*)
 - (v) How many bins did you choose and what were the size boundaries? Why? (*100 words max.*)
 - (vi) What is one benefit and one drawback of using a binned free list approach over a single free list? (*150 words max.*)

7 Hints

- Don't wait to implement `check_heap.c`. It is an essential tool that will catch bugs as they arise. Don't forget to use the flag that runs the heap checker.
- *Use the command-line options to runner.* During initial development, using tiny trace files will simplify debugging and testing. We have included several such trace files (look for file names starting with `short`) that you can use for initial debugging.
- *Compile the debug target first and use a debugger.* A debugger will help you isolate and identify out-of-bounds memory references.
- *Understand and use the data structures and getters/setters provided to you.* You are, of course, free to modify the data structures and functions, or ignore them entirely. However, they have been provided to assist you in your design and implementation, and it is certainly possible to get full credit (and earn additional points) without modifying the given helper functions and data structures.

- *Design, Design, Design!* There are many, many ways to build a dynamic memory allocator, and every design choice has benefits and tradeoffs. Before you commit to a design, fully understand what design tradeoffs you are making. It is a lot easier to debug code when you understand what you are planning to do. The write-up questions ask about some good design choices to think about, but they are not an exhaustive list of the design choices you will have to make.
- *Do your implementation in stages.* Start by doing a detailed system design and making sure that all the pieces will interact as intended. **Draw a picture of your heap!** If you come to us for help, we expect you to understand your design.
Next, implement a heap consistency checker—at least a simple version. Only then should you work on coding `umalloc` and `ufree` and making sure that they work correctly on the test traces. You can even defer coalescing until you have got the other pieces working. Performance and space optimization are always the final step; make sure you don't sacrifice correctness to gain efficiency.
- *Use a profiler.* You may find the `gprof` tool helpful for optimizing performance. You must learn and run this on your own, though. The teaching staff will not be available for assistance regarding `gprof`.
- *Start early!* It is possible to write an efficient memory allocator package in under 200 lines of code, if you do a good design up front. However, we can guarantee that it will be some of the most difficult and sophisticated code you have written so far in your career. So start early and good luck!



From <https://xkcd.com/138/>. Licensed under <https://xkcd.com/license.html>.